

LAB NO 13

DATA STRUCTURES AND ALGORITHMS

OBJECTIVE: Implementing Hashing techniques with collision resolution.

LAB TASKS

Task 1:

```
import java.util.Hashtable;

public class Main {
    public static void main(String[] args) {
        // Create a new Hashtable instance
        Hashtable<Integer, String> hashtable = new Hashtable<>();

        // Insert elements into the hashtable
        hashtable.put(1, "Abdullah");
        hashtable.put(2, "Affan");
        hashtable.put(3, "Messi");
        hashtable.put(4, "Neymar");
        hashtable.put(5, "Suarez");

        // Display the hashtable after insertion
        System.out.println("Hashtable after insertion: ");
        for (Integer key : hashtable.keySet()) {
            System.out.println("Key: " + key + " => Value: " + hashtable.get(key));
        }
    }
}
```

Output:

```
Hashtable after insertion:
Key: 5 => Value: Suarez
Key: 4 => Value: Neymar
Key: 3 => Value: Messi
Key: 2 => Value: Affan
Key: 1 => Value: Abdullah

=== Code Execution Successful ===
```

Task 2:

```
import java.util.HashMap;

public class Main {

    // HashMap to store vehicles with String keys and String values
    private HashMap<String, String> vehiclesMap;

    // Constructor to initialize the HashMap
    public Main() {
        vehiclesMap = new HashMap<>();
    }

    // Method to add a vehicle to the map
    public void add(String vehicleID, String vehicleName) {
        vehiclesMap.put(vehicleID, vehicleName);
        System.out.println("Vehicle added: " + vehicleID + " = " + vehicleName);
    }

    // Method to remove a vehicle from the map by vehicle ID
    public void remove(String vehicleID) {
        if (vehiclesMap.containsKey(vehicleID)) {
            vehiclesMap.remove(vehicleID);
            System.out.println("Vehicle with ID " + vehicleID + " has been removed.");
        } else {
            System.out.println("Vehicle with ID " + vehicleID + " not found.");
        }
    }

    // Method to search for a vehicle by vehicle ID
    public void search(String vehicleID) {
```

```
        public void display() {
            System.out.println("\nList of Vehicles:");
            for (String vehicleID : vehiclesMap.keySet()) {
                System.out.println("ID: " + vehicleID + " | Vehicle: " + vehiclesMap.get(
                    vehicleID));
            }
        }

        public static void main(String[] args) {
            // Create the Main object (which acts as Vehicles HashMap)
            Main vehicles = new Main();

            // Add some vehicles to the map
            vehicles.add("V001", "Tesla Model S");
            vehicles.add("V002", "BMW X5");
            vehicles.add("V003", "Audi A6");

            // Display all vehicles
            vehicles.display();

            // Search for a vehicle by ID
            vehicles.search("V002");

            // Remove a vehicle by ID
            vehicles.remove("V001");

            // Display all vehicles after removal
            vehicles.display();

            // Attempt to search for a removed vehicle
            vehicles.search("V001");
        }
    }
}
```

Output:

```
Vehicle added: V001 = Tesla Model S
Vehicle added: V002 = BMW X5
Vehicle added: V003 = Audi A6

List of Vehicles:
ID: V003 | Vehicle: Audi A6
ID: V002 | Vehicle: BMW X5
ID: V001 | Vehicle: Tesla Model S
Found vehicle: V002 = BMW X5
Vehicle with ID V001 has been removed.

List of Vehicles:
ID: V003 | Vehicle: Audi A6
ID: V002 | Vehicle: BMW X5
Vehicle with ID V001 not found.
```

Task 3:

```
import java.util.HashMap;

public class Main {

    public static int findMostFrequentElement(int[] arr) {
        HashMap<Integer, Integer> frequencyMap = new HashMap<>();

        for (int num : arr) {
            frequencyMap.put(num, frequencyMap.getOrDefault(num, 0) + 1);
        }
        int mostFrequentElement = arr[0];
        int maxFrequency = frequencyMap.get(mostFrequentElement);
        for (int key : frequencyMap.keySet()) {
            int frequency = frequencyMap.get(key);
            if (frequency > maxFrequency || (frequency == maxFrequency && key <
                mostFrequentElement)) {
                mostFrequentElement = key;
                maxFrequency = frequency;
            }
        }
        return mostFrequentElement;
    }

    public static void main(String[] args) {
        int[] arr = {10, 5, 10, 7, 10, 9, 5, 3, 10, 7, 10, 5};
        int result = findMostFrequentElement(arr);
        System.out.println("The most frequent element is: " + result);
    }
}
```

Output:

```
The most frequent element is: 10

=== Code Execution Successful ===
```

Task 4:

```
import java.util.HashMap;
public class Main {
    public static void calculateCumulativeSums(int[] arr) {
        HashMap<Integer, Integer> sumFrequencyMap = new HashMap<>();
        int cumulativeSum = 0;
        sumFrequencyMap.put(0, 1);

        for (int num : arr) {
            cumulativeSum += num;

            sumFrequencyMap.put(cumulativeSum, sumFrequencyMap.getOrDefault(
                cumulativeSum, 0) + 1);
        }

        System.out.println("Cumulative Sums and their Frequencies:");
        for (int sum : sumFrequencyMap.keySet()) {
            System.out.println("Sum: " + sum + " occurred " + sumFrequencyMap.get(sum)
                + " time(s).");
        }
    }

    public static void main(String[] args) {
        // Example array
        int[] arr = {1, -1, 2, 3, -2, 1};

        calculateCumulativeSums(arr);
    }
}
```

Output:

```
Cumulative Sums and their Frequencies:
Sum: 0 occurred 2 time(s).
Sum: 1 occurred 1 time(s).
Sum: 2 occurred 1 time(s).
Sum: 3 occurred 1 time(s).
Sum: 4 occurred 1 time(s).
Sum: 5 occurred 1 time(s).
```

```
=== Code Execution Successful ===
```

Task 5:

```
import java.util.LinkedList;

public class Main {

    // Define the initial capacity of the hash map
    private static final int INITIAL_CAPACITY = 16;

    // Define the table which will store the linked lists for separate chaining
    private LinkedList<Entry>[] table;

    // Constructor to initialize the table
    public Main() {
        table = new LinkedList[INITIAL_CAPACITY];
        for (int i = 0; i < INITIAL_CAPACITY; i++) {
            table[i] = new LinkedList<>();
        }
    }

    // Define the entry class which stores the key-value pairs
    private static class Entry {
        String key;
        String value;

        // Constructor
        Entry(String key, String value) {
            this.key = key;
            this.value = value;
        }
    }

    // Hash function to compute the index for a key
    private int getHash(String key) {

        public void put(String key, String value) {
            int index = getHash(key);
            LinkedList<Entry> bucket = table[index];

            // Check if the key already exists in the linked list, and update the value
            // if found
            for (Entry entry : bucket) {
                if (entry.key.equals(key)) {
                    entry.value = value; // Update the value for the existing key
                    return;
                }
            }

            // If the key doesn't exist, add a new entry
            bucket.add(new Entry(key, value));
        }

        // Method to get the value associated with a key
        public String get(String key) {
            int index = getHash(key);
            LinkedList<Entry> bucket = table[index];

            // Search for the key in the linked list and return the corresponding value
            for (Entry entry : bucket) {
                if (entry.key.equals(key)) {
                    return entry.value;
                }
            }

            return null; // Return null if the key is not found
        }
    }
}
```

```
public void remove(String key) {
    int index = getHash(key);
    LinkedList<Entry> bucket = table[index];

    // Search for the key in the linked list and remove the corresponding entry
    for (Entry entry : bucket) {
        if (entry.key.equals(key)) {
            bucket.remove(entry);
            return;
        }
    }
}

// Method to display the contents of the hash map
public void display() {
    for (int i = 0; i < table.length; i++) {
        LinkedList<Entry> bucket = table[i];
        if (!bucket.isEmpty()) {
            System.out.print("Bucket " + i + ": ");
            for (Entry entry : bucket) {
                System.out.print "[" + entry.key + "=" + entry.value + " ] ";
            }
            System.out.println();
        }
    }
}

// Main method to test the custom HashMap
public static void main(String[] args) {
    Main map = new Main();
```

```
    map.put("Tesla", "Electric Car");
    map.put("Einstein", "Physicist");
    map.put("Amazon", "E-commerce Giant");
    map.put("Python", "Programming Language");

    // Displaying the map
    map.display();

    // Retrieving a value using key
    System.out.println("Value for 'Einstein': " + map.get("Einstein"));

    // Updating the value of an existing key
    map.put("Python", "Snake");
    System.out.println("Updated Value for 'Python': " + map.get("Python"));

    // Removing a key-value pair
    map.remove("Amazon");

    // Display the map after removal
    System.out.println("After removing 'Amazon':");
    map.display();
}
```

Output:

```
Bucket 4: [Amazon=E-commerce Giant] [Python=Programming Language]
Bucket 7: [Tesla=Electric Car]
Bucket 15: [Einstein=Physicist]
Value for 'Einstein': Physicist
Updated Value for 'Python': Snake
After removing 'Amazon':
Bucket 4: [Python=Snake]
Bucket 7: [Tesla=Electric Car]
Bucket 15: [Einstein=Physicist]

=== Code Execution Successful ===
```

HomeTask:

```
import java.util.LinkedList;

public class Main {

    // Define the initial capacity of the hash table
    private static final int TABLE_SIZE = 11;

    // Define the hash table, each bucket will hold a LinkedList to handle
    // collisions
    private LinkedList<Player>[] table;

    // Constructor to initialize the hash table
    public Main() {
        table = new LinkedList[TABLE_SIZE];
        for (int i = 0; i < TABLE_SIZE; i++) {
            table[i] = new LinkedList<>();
        }
    }

    // Player class to store name and runs
    private static class Player {
        String name;
        int runs;

        // Constructor for Player
        Player(String name, int runs) {
            this.name = name;
            this.runs = runs;
        }
    }
}
```

```
private int getHash(int runs) {
    return runs % TABLE_SIZE;
}

// Method to insert a player into the hash table
public void insertPlayer(String name, int runs) {
    int index = getHash(runs);
    LinkedList<Player> bucket = table[index];

    // Check if the player with the same runs exists, if so update
    for (Player player : bucket) {
        if (player.name.equals(name)) {
            player.runs = runs; // Update the runs if the player already exists
            return;
        }
    }

    // If the player is not found, add a new entry
    bucket.add(new Player(name, runs));
}

// Method to get the rank of a player (the index position in the hash table)
public String getRankByRuns(int runs) {
    int index = getHash(runs);
    LinkedList<Player> bucket = table[index];

    // Search for the player with the given runs in the bucket
    for (Player player : bucket) {
        if (player.runs == runs) {
            return "Player with runs " + runs + " is ranked at index " + index
                + " in the hash table.";
        }
    }
}
```

```
        System.out.println();
    }
}

public static void main(String[] args) {
    Main cricketTeam = new Main();

    // Adding cricket players and their runs
    cricketTeam.insertPlayer("Player1", 34);
    cricketTeam.insertPlayer("Player2", 58);
    cricketTeam.insertPlayer("Player3", 30);
    cricketTeam.insertPlayer("Player4", 22);
    cricketTeam.insertPlayer("Player5", 77);
    cricketTeam.insertPlayer("Player6", 45);
    cricketTeam.insertPlayer("Player7", 65);
    cricketTeam.insertPlayer("Player8", 31);
    cricketTeam.insertPlayer("Player9", 52);
    cricketTeam.insertPlayer("Player10", 41);
    cricketTeam.insertPlayer("Player11", 60);

    // Display the hash table with players and their runs
    cricketTeam.display();

    // Find out the rank of a player based on their runs
    System.out.println(cricketTeam.getRankByRuns(30)); // Rank of Player3 (30
    runs)
    System.out.println(cricketTeam.getRankByRuns(77)); // Rank of Player5 (77
    runs)
    System.out.println(cricketTeam.getRankByRuns(45)); // Rank of Player6 (45
    runs)
}
```


Output:

```
Rank #0: Player4 (22 runs) Player5 (77 runs)
Rank #1: Player1 (34 runs) Player6 (45 runs)
Rank #3: Player2 (58 runs)
Rank #5: Player11 (60 runs)
Rank #8: Player3 (30 runs) Player9 (52 runs) Player10 (41 runs)
Rank #9: Player8 (31 runs)
Rank #10: Player7 (65 runs)
Player with runs 30 is ranked at index 8 in the hash table.
Player with runs 77 is ranked at index 0 in the hash table.
Player with runs 45 is ranked at index 1 in the hash table.
```

HomeTask:

```
import java.util.HashMap;
import java.util.Map;

public class Main {

    // HashMap to store page URL as key and visit count as value
    private HashMap<String, Integer> pageVisits;

    // Constructor to initialize the pageVisits HashMap
    public Main() {
        pageVisits = new HashMap<>();
    }

    // Method to increment the visit count for a page
    public void visitPage(String pageUrl) {
        // If the page already exists, increment the visit count, otherwise add the
        // page with a count of 1
        pageVisits.put(pageUrl, pageVisits.getOrDefault(pageUrl, 0) + 1);
    }

    // Method to retrieve the visit count for a specific page
    public int getVisitCount(String pageUrl) {
        return pageVisits.getOrDefault(pageUrl, 0);
    }

    // Method to get the page with the highest visit count
    public String getPageWithHighestVisits() {
        if (pageVisits.isEmpty()) {
            return null; // Return null if there are no pages visited
        }

        String mostVisitedPage = null;
```

```
    }

    System.out.println("Page Visits:");
    for (Map.Entry<String, Integer> entry : pageVisits.entrySet()) {
        System.out.println("Page: " + entry.getKey() + " | Visits: " + entry
            .getValue());
    }
}

// Main method to test the functionality of the Main class
public static void main(String[] args) {
    // Create the Main object
    Main analytics = new Main();

    // Simulate some page visits
    analytics.visitPage("https://www.example.com/home");
    analytics.visitPage("https://www.example.com/about");
    analytics.visitPage("https://www.example.com/contact");
    analytics.visitPage("https://www.example.com/home");
    analytics.visitPage("https://www.example.com/home");

    // Display all pages and their visit counts
    analytics.displayAllVisits();

    // Retrieve the visit count for a specific page
    System.out.println("\nVisit count for 'https://www.example.com/home': " +
        analytics.getVisitCount("https://www.example.com/home"));

    // Retrieve the page with the highest visit count
    System.out.println("\nPage with the highest visit count: " +
        analytics.getPageWithHighestVisits());
}
```

Output:

```
Page Visits:
Page: https://www.example.com/about | Visits: 1
Page: https://www.example.com/home | Visits: 3
Page: https://www.example.com/contact | Visits: 1

Visit count for 'https://www.example.com/home': 3

Page with the highest visit count: https://www.example.com/home
```